Building a Chatbot with LangGraph

Persistence, Memory, Checkpointers & Threads

Comprehensive Lecture Notes

Based on CampusX Lecture Series

TABLE OF CONTENTS

01	**Introduction & Roadmap**	*What we're building and why*
02	**The Problem: Statelessness**	*Why chatbots forget everything*
03	**Core Concepts of Persistence**	*Reducers, Checkpointers & Threads*
04	**The add_messages Reducer**	*Accumulating conversation history*
05	**Checkpointers Deep Dive**	*MemorySaver vs Database backends*
06	**Threads & Session Management**	*Multi-user conversation isolation*
07	**The Persistent Workflow**	*Step-by-step data flow explained*
08	**Code Implementation**	*Putting theory into practice*
09	**Advanced Concepts**	*Production tips and best practices*
10	**Quick Reference & Summary**	*Everything at a glance*

01 Introduction & Roadmap

This lecture focuses on building a **production-grade Chatbot** using LangGraph. While previous lectures covered basic workflow patterns (Sequential, Parallel, Conditional), this session dives into the most critical feature any chatbot needs: **conversational memory**.

What is LangGraph?

LangGraph is a framework built on top of LangChain for creating **stateful, multi-step** AI agent workflows. It models your application as a **directed graph** where nodes represent actions (like calling an LLM or a tool) and edges define the flow of execution. Unlike simple chain-based approaches, LangGraph supports cycles, branching, and persistent state management.

The Complete Chatbot Roadmap:

- **Memory & Persistence** — Remembering past conversations across turns (Today's Focus)
- **RAG (Retrieval Augmented Generation)** — Answering questions from external documents and knowledge bases
- **Tool Usage** — Searching the web, performing calculations, calling APIs
- **Human-in-the-Loop** — Asking for user confirmation before executing critical actions

Key Insight

Why start with Memory? Without persistence, a chatbot is just a stateless Q&A engine. Memory is the foundation that transforms isolated interactions into coherent conversations. Every other feature (RAG, Tools, HITL) builds on top of this persistent state.

02 The Problem: Statelessness

When you build a simple LangGraph workflow and run it in a conversational loop, you'll quickly discover a critical flaw: **the bot has no memory**. Each invocation is completely independent of the previous one, as if talking to a stranger every single time.

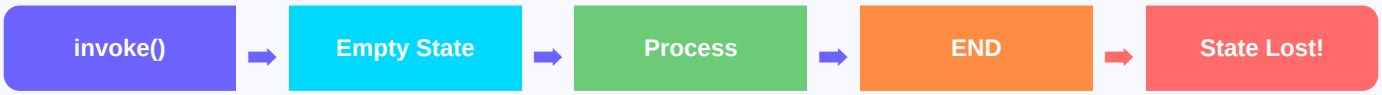
■ The "Amnesia" Demonstration

Turn	User Says	Bot Responds	Why?
1	"My name is Nitish"	"Hello Nitish!"	Current input is sufficient
2	"What is my name?"	"I don't know"	No memory of Turn 1!

Root Cause Analysis

- **Stateless Execution:** Standard LLM workflows have no built-in memory. Each API call is independent.
- **Fresh State Object:** Every `graph.invoke()` creates a brand-new, empty State dictionary.
- **No Data Carry-Over:** When execution reaches the END node, all intermediate data is discarded.
- **Isolation by Design:** This is actually intentional in most API designs for simplicity and scalability.

Stateless Flow (Without Persistence):



■ Analogy

Think of it like the movie character **Ghajini** — the bot has short-term memory loss. Every conversation turn starts from absolute zero with no recollection of what happened before.

03 Core Concepts of Persistence

To transform our stateless bot into a conversational agent, LangGraph provides three interconnected mechanisms that work together to create persistent memory:

A	B	C
add_messages Reducer	**Checkpointers**	**Threads**
Tells the state HOW to combine old & new messages	Saves state snapshots to external storage (RAM or Database)	Isolates conversations per user/session using unique IDs

The Connection

How they work together: The Reducer defines the merge strategy, the Checkpointer handles save/load operations, and Threads ensure each user gets their own isolated history. Remove any one of these, and the system breaks.

04 The add_messages Reducer

In LangGraph, a **Reducer** is a function that defines how new data merges with existing data in the state. By default, new values **overwrite** old values. This is fine for simple data but catastrophic for conversation history.

Overwrite vs. Append:

Without Reducer (Default)	With add_messages Reducer
State = [Msg1] New = [Msg2] **Result = [Msg2]** Msg1 is DELETED!	State = [Msg1] New = [Msg2] **Result = [Msg1, Msg2]** History is PRESERVED!

Implementation in Code:

```
from langgraph.graph import MessagesState
from langchain_core.messages import AnyMessage
from typing import Annotated
from langgraph.graph import add_messages

# Method 1: Using Annotated type hint
class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

# Method 2: Using built-in MessagesState (shorthand)
# MessagesState already includes the add_messages reducer
```

The `Annotated` type hint is Python's way of attaching metadata to a type. Here it tells LangGraph: *"This field is a list, and whenever you update it, use the add_messages function instead of simple assignment."*

How the Message List Grows:

Turn	State After Turn
1	`[HumanMessage("My name is Nitish"), AIMessage("Hello Nitish!")]`
2	`[HumanMessage("My name is Nitish"), AIMessage("Hello Nitish!"), HumanMessage("What is my name?"), AIMessage("Your name is Nitish!")]`
3	`[...all previous messages..., HumanMessage("Thanks!"), AIMessage("You're welcome!")]`

05 Checkpointers Deep Dive

A **Checkpointer** is the persistence layer that sits between your graph execution and the storage backend. It automatically takes **snapshots** of the entire state at every step of the workflow, enabling replay, time-travel debugging, and conversation continuity.

Types of Checkpointers:

■ MemorySaver (In-Memory)

Storage: Computer's RAM (in-process memory)

Speed: Extremely fast (no I/O overhead)

Persistence: Lost when the Python process ends

Use Case: Development, testing, prototyping

Setup: `from langgraph.checkpoint.memory import MemorySaver`

■ Database Checkpointers (Production)

Storage: Persistent database on disk or remote server

Speed: Slightly slower (network/disk I/O)

Persistence: Survives server restarts, deployments, crashes

Use Case: Production applications with real users

Options: PostgreSQL (recommended), SQLite, Redis, MongoDB

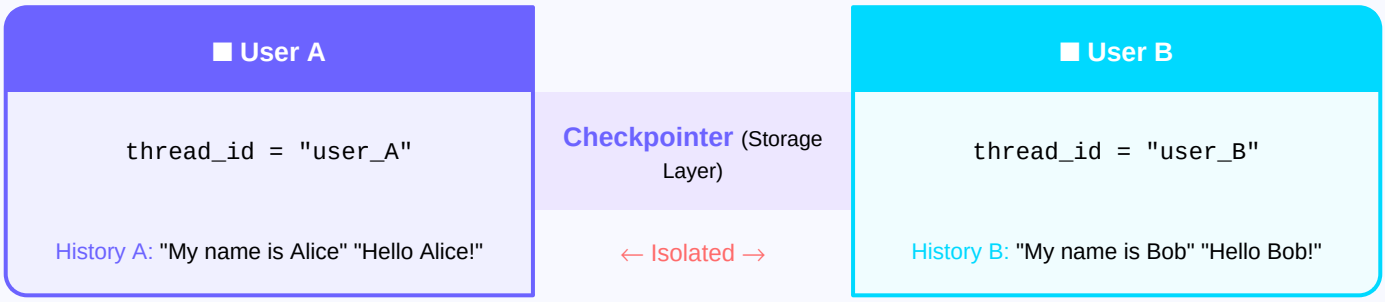
Comparison at a Glance:

Feature	MemorySaver	PostgresSaver	SqliteSaver
Speed	■ Fastest	Fast	Fast
Survives Restart	✘ No	✔ Yes	✔ Yes
Multi-Process	✘ No	✔ Yes	Limited
Scalable	✘ No	✔ Yes	Moderate
Best For	Dev/Testing	**Production**	Small Apps

06 Threads & Session Management

In production, a chatbot serves **multiple users simultaneously**. The system needs a way to keep each user's conversation history completely separate. This is where **Threads** come in.

How Thread IDs Work:



Implementation:

```
# Configuration with thread ID
config = {"configurable": {"thread_id": "user_123"}}

# Each invoke uses the config to load/save the right history
result = graph.invoke(
    {"messages": [HumanMessage(content="Hello!")]},
    config=config
)
```

■ Thread Strategies

Common Thread ID Strategies:

- **Per User:** `thread_id = user_id` — One continuous conversation per user
- **Per Session:** `thread_id = session_id` — New thread each login
- **Per Topic:** `thread_id = f"{user}_{topic}"` — Separate threads per subject
- **UUID-Based:** `thread_id = uuid4()` — Unique per conversation

07 The Persistent Workflow

Once all three components (Reducer, Checkpointer, Thread) are in place, the data flow fundamentally changes. Here is the **step-by-step execution** of a persistent chatbot:

1

■ User Input

The user types their message, e.g. "What is my name?"

2

■ Config Check

The code sends the input along with a config dict containing the thread_id

3

■ State Loading

LangGraph queries the Checkpointer for the saved state associated with this thread_id. Instead of starting empty, it loads the FULL conversation history!

4

■ LLM Processing

The LLM receives the complete context: all previous messages + the new question. It can now understand the full conversation.

5

■ Generation

With full context, the LLM generates an informed response: "Your name is Nitish!"

6

■ State Saving

The Checkpointer saves the updated state (with new messages appended) back to storage, ready for the next turn.

■ Key Takeaway

The Magic is in Step 3! This is where persistence changes everything. Instead of a blank slate, the graph loads the entire conversation context. The LLM doesn't even know it's being "restarted" — from its perspective, it's one continuous conversation.

08 Code Implementation

Here is the complete implementation that brings all the concepts together into a working persistent chatbot:

Step 1: Define the State with Reducer

```
from typing import Annotated, TypedDict
from langchain_core.messages import AnyMessage
from langgraph.graph import add_messages

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Step 2: Create the Chatbot Node

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4")

def chatbot(state: State):
    # The LLM receives ALL messages (history + new)
    response = model.invoke(state["messages"])
    return {"messages": [response]}
```

Step 3: Build the Graph with Checkpointer

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

# Initialize checkpointer
memory = MemorySaver()

# Build graph
graph_builder = StateGraph(State)
graph_builder.add_node("chatbot", chatbot)
graph_builder.add_edge(START, "chatbot")
graph_builder.add_edge("chatbot", END)

# Compile WITH checkpointer
graph = graph_builder.compile(checkpointer=memory)
```

Step 4: Run with Thread ID

```
from langchain_core.messages import HumanMessage

config = {"configurable": {"thread_id": "session_001"}}

# Turn 1
response = graph.invoke(
```

```
    {"messages": [HumanMessage(content="My name is Nitish")]},
    config=config
)
# Output: "Hello Nitish!"

# Turn 2 - SAME thread_id
response = graph.invoke(
    {"messages": [HumanMessage(content="What is my name?")]},
    config=config
)
# Output: "Your name is Nitish!" ■ Memory works!
```

09 Advanced Concepts

A. State Snapshots & Time-Travel Debugging

Since the Checkpointer saves state at **every step** (not just at the end), you can inspect or replay any intermediate state. This is incredibly powerful for debugging complex multi-step agent workflows. You can literally "rewind" to any point in the conversation.

```
# Get all checkpoints for a thread
checkpoints = memory.list(config)
for cp in checkpoints:
    print(cp.metadata) # Inspect each snapshot
```

B. Message Trimming & Summarization

As conversations grow, the message list becomes very long. LLMs have token limits, and long histories increase cost and latency. Solutions include:

- **Window-based Trimming:** Keep only the last N messages (e.g., last 20 turns). Simple but loses old context.
- **Token-based Trimming:** Keep messages that fit within a token budget. More precise control over LLM input size.
- **Summarization:** Periodically summarize old messages into a single summary message. Best of both worlds — retains key context while staying compact.
- **Hybrid Approach:** Keep a summary of old messages + full recent messages. Used by most production chatbots.

C. Graph Architecture Patterns

Pattern	Flow	Description
Simple Chatbot	`START → LLM Node → END`	Basic Q&A; with memory
RAG Chatbot	`START → Retriever → LLM → END`	Answer from documents
Agent with Tools	`START → LLM → [Tool?] → LLM → END`	Web search, calculator, etc.
HITL Agent	`START → LLM → Human Check → Tool → END`	User confirms actions

■ Pro Tip

Production Tip: In real-world applications, always use a **database-backed checkpointer** (PostgreSQL recommended). Combine it with **message summarization** to handle long conversations efficiently. Set up **monitoring** on your checkpoint storage to track memory usage and conversation lengths over time.

10 Quick Reference & Summary

Core Concepts at a Glance:

Concept	What It Does	Key Code
State	Holds all conversation data during execution	`class State(TypedDict):`
Reducer	Defines merge strategy (append vs overwrite)	`Annotated[list, add_messages]`
Checkpointer	Saves/loads state snapshots to storage	`MemorySaver() / PostgresSaver()`
Thread	Isolates conversations per user/session	`{"thread_id": "user_123"}`
Graph	Orchestrates nodes, edges, and execution flow	`StateGraph(State).compile()`

The Big Picture

State is usually **transient** (temporary) — it exists only during a single graph execution. **Checkpointers** make it **persistent** (permanent) by saving snapshots to an external storage system (RAM or Database), keyed by a **Thread ID**.

*This single architectural pattern — persist, reload, append — is what transforms a one-off task execution engine into a true **conversational AI agent**.*

Reducer (add_messages) + **Checkpointer** (MemorySaver) + **Thread ID** (session key) = **Persistent Chatbot!**